

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Bogdan Aramă FAF-242

Report

Laboratory work n.1

of Cryptography

Verified by:
Zaica Maia

Chisinau – 2026

1 Objective 3
2 Short theory 3
2.1 Caesar cipher 3
2.2 Caesar cipher with a permutation 3
3 Implementation 4
3.1 Alphabet Enconding and Normalization 4
3.2 Permuted Alphabet Construction 4
3.3 Core Modular Arithmetic Engine 5
4 Results and Verification 6
Standard Caesar Cipher Execution 6
Caesar Cipher with Permutation Execution 6
5 Conclusions 6
6 Source Code 6

Caesar Cipher

1 Objective

The objective is to implement encryption and decryption for the 31-letter Romanian alphabet using the Caesar cipher and a keyword-based permutation. The program demonstrates reversible letter substitution, validates user input, and shows how changing the alphabet changes the ciphertext.

2 Short theory

2.1 Caesar cipher

The Caesar cipher is an elementary symmetric monoalphabetic substitution cipher where each letter in the plaintext is replaced by a character located a fixed distance k positions down the alphabet. For an alphabet of size n, the secret key k satisfies k in {1, 2, ..., n - 1}, excluding k = 0 as it produces an identity mapping.

Mathematically, encryption and decryption are defined over the ring of integers modulo n as:

- **Encryption: $c = e_k(x) = (x + k) \bmod n$**
- **Decryption: $m = d_k(y) = (y - k) \bmod n$**

where x, y represent the zero-based numeric codes of the plaintext and ciphertext characters, respectively.

For the Romanian alphabet with n = 31 letters (A, Ă, Â, B, ..., Z), the total key space is |K| = n - 1 = 30. Because of this small key space, the cipher is vulnerable to brute-force cryptanalysis, where an attacker can test all 30 candidate shifts.

2.2 Caesar cipher with a permutation

To increase resistance against brute-force attacks, the underlying sequence of the alphabet is permuted using a keyword (Key 2) prior to applying the Caesar shift (Key 1). The permuted sequence is constructed by extracting the unique letters of the keyword in order of appearance, followed by the remaining characters of the Romanian alphabet in their natural order.

The Caesar shift k1 in {1, ..., 30} is then evaluated based on character indices within this permuted sequence. For an alphabet of size n = 31, there are 31! ≈ 8.2228 × 10^33 possible alphabet arrangements, giving an expanded nominal key space:

$K_{total} = 31! \times 30 \approx 2.4668 \times 10^{35}$ possible keys

While this prevents an exhaustive key search, the system remains a monoalphabetic substitution cipher: each plaintext character maps to an identical ciphertext character across the message. Consequently, it remains vulnerable to frequency analysis.

3 Implementation

The system is structured into three Java modules: RomanianAlphabet.java, CaesarCipher.java, and Main.java. The implementation avoids built-in ASCII/Unicode shifts, relying on the 31-letter Romanian alphabet definition.

3.1 Alphabet Enconding and Normalization

The 31 Romanian characters are defined using Unicode escape literals to ensure cross-platform compatibility across Windows command lines. The normalization routine cleans whitespace, translates text to uppercase, and maps cedilla-based characters (Ş, Ţ) to standard comma-below variants (Ș, Ț):

```
public static final String ALPHABET_STRING = "A\u0102\u00C2BCDEFGHI\u00CEJKLMNOPQRS\u0218T\u021AUVWXYZ";
public static final int ALPHABET_SIZE = ALPHABET_STRING.length(); // 31
```

```
public static String normalizeText(String input) {
    if (input == null) return "";

    // Normalize cedillas (Ş, ş, Ţ, ţ) to standard comma-below (Ș, ț)
    String text = input.replace('\u015E', '\u0218') // Ş -> Ș
                       .replace('\u015F', '\u0219') // ş -> ș
                       .replace('\u0162', '\u021A') // Ţ -> Ț
                       .replace('\u0163', '\u021B'); // ţ -> ț

    // Convert to uppercase
    text = text.toUpperCase();

    // Remove all whitespace
    return text.replaceAll("\\s+", "");
}
```

3.2 Permuted Alphabet Construction

The permutation algorithm iterates over the normalized keyword (minimum 7 letters), inserting unique characters into a Set and StringBuilder, before appending the remaining unused alphabet characters in their natural order:

```java
public static String buildPermutedAlphabet(String keyword) {
    StringBuilder sb = new StringBuilder();
    Set<Character> seen = new HashSet<>();

    // Add distinct characters from the normalized keyword
    for (char c : keyword.toCharArray()) {
        if (!seen.contains(c)) {
            seen.add(c);
            sb.append(c);
        }
    }

    // Add remaining characters in their natural Romanian order
    for (char c : ALPHABET_STRING.toCharArray()) {
        if (!seen.contains(c)) {
            seen.add(c);
            sb.append(c);
        }
    }

    return sb.toString();
}
```

3.3 Core Modular Arithmetic Engine

Encryption applies (x + k) mod n, while decryption resolves negative modulo outputs via (y - k) mod n + n:

```java
public static String encrypt(String text, int key, String alphabet) {
    int n = alphabet.length();
    StringBuilder cipherText = new StringBuilder();

    for (char c : text.toCharArray()) {
        int x = RomanianAlphabet.getIndex(c, alphabet);
        int encIndex = (x + key) % n;
        cipherText.append(RomanianAlphabet.getChar(encIndex, alphabet));
    }

    return cipherText.toString();
}
```

```java
public static String decrypt(String text, int key, String alphabet) {
    int n = alphabet.length();
    StringBuilder plainText = new StringBuilder();

    for (char c : text.toCharArray()) {
        int y = RomanianAlphabet.getIndex(c, alphabet);
        int decIndex = (y - key) % n;
        if (decIndex < 0) {
            decIndex += n; // Wrap around negative modulo
        }
        plainText.append(RomanianAlphabet.getChar(decIndex, alphabet));
    }

    return plainText.toString();
}
```

4. Results and Verification

Below are documented execution runs demonstrating both tasks, diacritic validation, and error-handling conditions.

4.1 Task 1.1: Standard Caesar Cipher Execution

```
--- Task 1.1: Standard Caesar Cipher ---
Select operation (1 = Encrypt, 2 = Decrypt): 1
Enter shift key k (1-30): 4
Enter plaintext: ASTAZI PLECAM LA MUNTE

[Result] Normalized Plaintext : ASTAZIPLECAMLAMUNTE
[Result] Ciphertext           : CUWCBLȘPIGCQPCQYRWI

Select Task:
1. Task 1.1 - Standard Caesar Cipher (1 Key)
2. Task 1.2 - Caesar Cipher with Permutation (2 Keys)
0. Exit
Choice: 1

--- Task 1.1: Standard Caesar Cipher ---
Select operation (1 = Encrypt, 2 = Decrypt): 2
Enter shift key k (1-30): 4
Enter ciphertext: CUWCBLȘPIGCQPCQYRWI

[Result] Normalized Ciphertext: CUWCBLȘPIGCQPCQYRWI
[Result] Decrypted Plaintext  : ASTAZIPLECAMLAMUNTE
```

4.2 Task 1.2: Caesar Cipher with Permutation Execution

```
--- Task 1.2: Caesar Cipher with Permutation ---
Select operation (1 = Encrypt, 2 = Decrypt): 1
Enter shift key k1 (1-30): 5
Enter Key 2 (keyword, >= 7 letters): unstoppable

[Generated Permuted Alphabet]:
UNSTOPABLEĂÂCDFGHIÎJKMQRȘȚVWXYZ
Enter plaintext: VITEZA LUMINII ESTE FOARTE MARE

[Result] Normalized Plaintext : VITEZALUMINIIESTEFOARTEMARE
[Result] Ciphertext           : UQLFOÂDPVQAQQFBLFJEÂXLFVÂXF
```

5. Conclusions

In this laboratory work, the Caesar cipher was analyzed and implemented across two variants using the Romanian alphabet (n = 31). The key findings of this work include:

1. Key Space Limitations: The standard Caesar cipher provides negligible security due to its small key space of only 30 possible keys. An adversary can recover the plaintext via an exhaustive search in milliseconds.

2. Effect of Permutation: Introducing a keyword permutation (Key 2) expands the theoretical key space to 31! × 30 ≈ 2.47 × 10^35 keys, eliminating the feasibility of an exhaustive brute-force attack.

3. Monoalphabetic Vulnerability: Despite the large key space, the keyword Caesar cipher remains a monoalphabetic substitution cipher. Because each letter maps to a single ciphertext equivalent throughout the message, the letter frequency distribution of the Romanian language is preserved, allowing the cipher to be broken through frequency analysis.

6. Source Code Repository

The complete source code is available in the GitHub repository:
https://github.com/BogdanArama/CS-Lab1-CaesarCipher